

Distributed Transactions

Saga Pattern · Choreography · Kafka · Compensation



Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

DAY

Wednesday

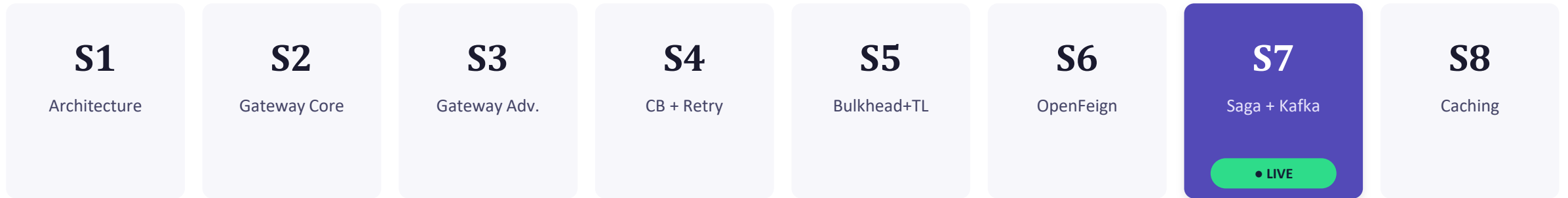
DURATION

2.5 Hours · Online

PHASE

Phase 1 — Foundation

Phase 1 — Foundation & Core Patterns



Today: the most architecturally important session in Phase 1 — where partial failure becomes business inconsistency, and Choreography Sagas restore it.

Full 29-session roadmap: see Course Kickoff deck

Sync Calls Work — Until They Partially Fail

WHAT WE HAVE, SESSIONS 1–6

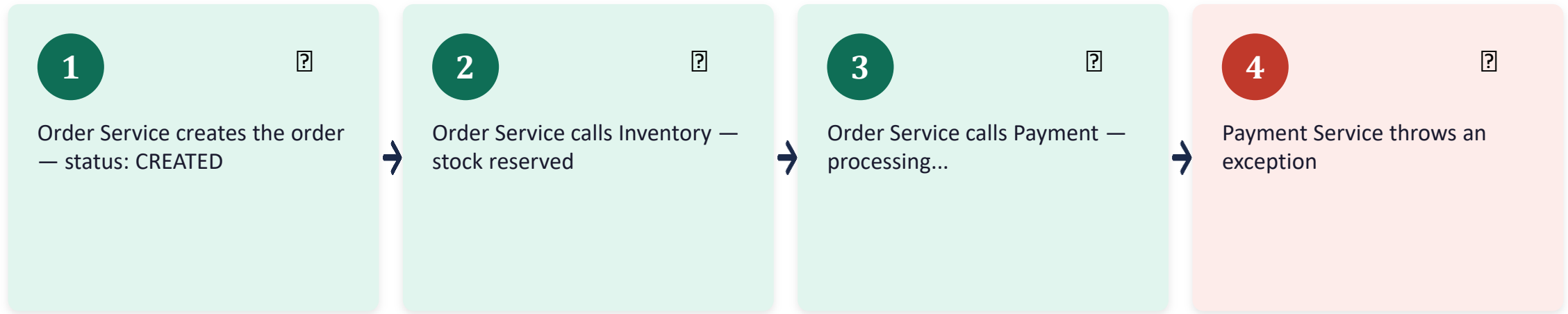
- Gateway secured with JWT + Rate Limiting
- Order Service: full Resilience4j stack (CB, Retry, Bulkhead, TimeLimiter)
- Inventory Service added — Order calls it synchronously via OpenFeign
- Every call so far: Order waits, gets an answer, decides



But what happens when ONE step succeeds and the NEXT one fails?

- Order created ?
- Inventory reserved ?
- Payment fails ?
- Resilience4j (CB/Retry) handles THIS call's failure — but who fixes the order and the reservation that already happened?

A Customer Places an Order for 3 Units of PROD-001



Now what?

- The order is CREATED in the Order DB. 3 units are RESERVED in the Inventory DB. The payment FAILED — money was not charged.
- Next customer tries to buy PROD-001 — shows out of stock — but nothing was sold!

Business Inconsistency

WHAT IT IS NOT

- Not a code bug — nothing threw an unhandled exception
- Not a network error — every call got a real response
- Not something try/catch was supposed to fix

WHAT IT IS

A distributed state problem: each service did its own job correctly, but the SYSTEM as a whole is left in a state that violates a business rule.

-
- Order DB says: CREATED
 - Inventory DB says: 3 units RESERVED
 - Payment DB says: nothing charged
 - No single database is "wrong" — but together, they describe a system that makes no business sense

Monolith: ACID Transactions Just Work



One database. One transaction. All or nothing.

```
BEGIN TRANSACTION
  INSERT order
  UPDATE inventory
  UPDATE payment
COMMIT    <- all or nothing

If payment fails:
  ROLLBACK    <- all undone
```

WHY THIS WORKS

- Single connection, single coordinator
- Database engine guarantees atomicity natively
- Failure anywhere → the whole transaction reverses cleanly
- Simple. Reliable. Boring — in a good way.

Why ACID Fails Across Microservices?

□ Monolith — ACID Works

Single Database

```
BEGIN TRANSACTION
  INSERT order
  UPDATE inventory
  UPDATE payment
COMMIT ← all or nothing
```

If payment fails:
ROLLBACK ← all undone

Simple. Reliable.
One DB. One transaction.

□ Microservices — 2PC Fails in Practice

Three separate databases

```
BEGIN DISTRIBUTED TX
  Order DB → INSERT order
  Inventory DB → UPDATE stock
  Payment DB → UPDATE balance
COMMIT ← who coordinates this?
```

Problems:

- × Coordinator is single point of failure
- × All services must be UP to commit
- × Locks held across network → slow
- × Does not scale horizontally

Microservices: Three Separate Databases



No shared connection. No single coordinator available at scale.



Order DB

`INSERT order`



Inventory DB

`UPDATE stock`



Payment DB

`UPDATE balance`

BEGIN DISTRIBUTED TX → COMMIT ← who coordinates this?

- Coordinator is a single point of failure × All services must be UP to commit
- Locks held across network = slow × Does not scale horizontally

Why 2PC Is Impractical at Microservices Scale



Single Point of Failure

If the coordinator crashes mid-transaction, services are stuck holding locks.



All Services Must Be Up

If Inventory is down, the entire transaction cannot even start.



Network Locks

Services hold DB locks while waiting for the coordinator — kills throughput.



Does Not Scale

Coordination overhead grows polynomially as services are added.

The Industry Conclusion

- 2PC is a theoretical solution that does not work at scale.
- The practical answer: accept eventual consistency + use the Saga pattern for compensation.

What Is a Saga?



A sequence of local transactions, coordinated through events.
Each step updates its own service's database, then publishes an event to trigger the next step.

IF ANY STEP FAILS

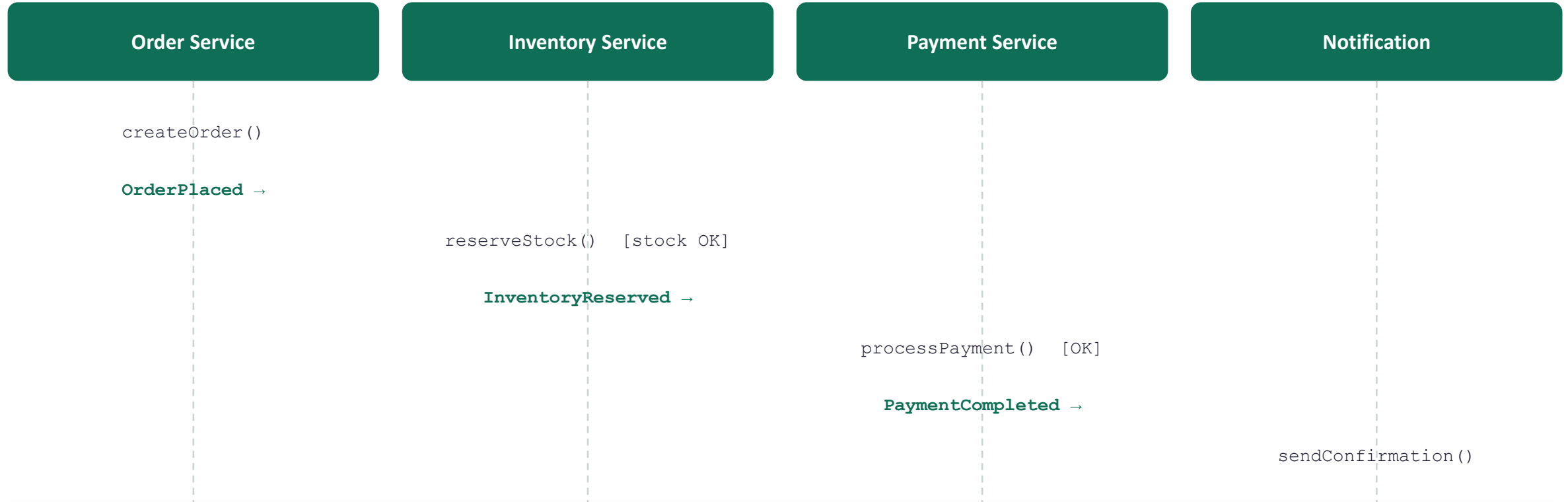
- The Saga publishes a failure event
- All COMPLETED previous steps execute their COMPENSATION transaction
- The system reaches a NEW consistent state — not the original state

Key Insight

There is no global ROLLBACK.

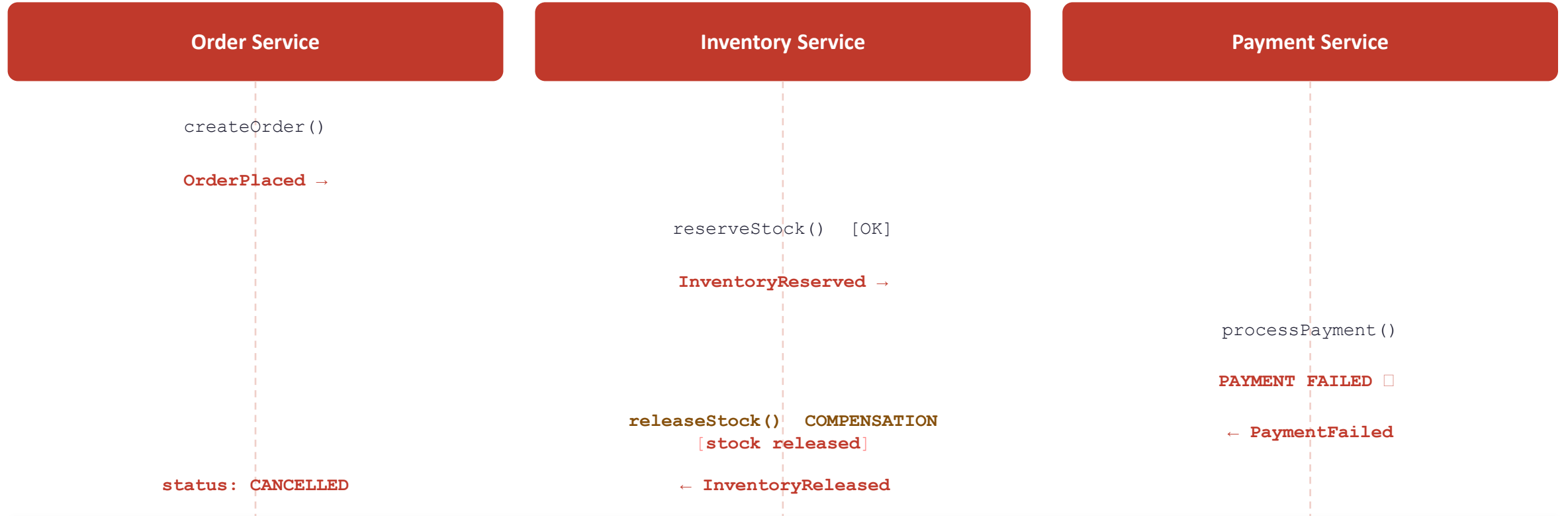
Each service is responsible for undoing its own work if the Saga fails.

Order → Inventory → Payment → Confirmed



status: CONFIRMED — each arrow is a Kafka event. Each service handles only its own local DB transaction.

Payment Fails → Release Stock → Cancel Order



status: Order CANCELLED, Stock RELEASED — No ROLLBACK. Each service executes a compensation transaction — a new consistent state.



THE GOLDEN CONCEPT

COMPENSATE $\neq$ ROLLBACK

ROLLBACK

*Means it never happened. Reverts to the original state
— only possible inside a single ACID transaction.*

COMPENSATE

*Acknowledges what happened and takes corrective
action. The Payment Service does not undo the charge
— it issues a refund.*



MANDATORY WORKSHOP

Draw the Saga on Paper

⚠️ Do not skip or rush this. If trainees cannot draw the Saga, they cannot implement it. (10 min work + 5 min debrief)

1 Happy Path

Draw the event sequence from “Order placed” to “Order confirmed.” Label each event. Show which service publishes and which consumes.

2 Compensation Path

What happens after PaymentFailed? Which service compensates? What does compensation look like? Final DB state of each service?

3 Bonus: Inventory Down

Draw what happens if Inventory Service is DOWN when OrderPlaced is published. How long does the message wait?

Choreography vs Orchestration

	Choreography	Orchestration
Coordinator	No central coordinator — events drive the flow	Central Saga orchestrator service
Coupling	Services only know about events — loosely coupled	Services know about the orchestrator
Debugging	Harder — must trace events across services	Easier — one place to see the full flow
Scalability	High — no bottleneck	Lower — orchestrator can become a bottleneck

- Today's use:  We implement CHOREOGRAPHY
- Orchestration is discussed in Session 12 — keep this badge in mind for that day too

What Kafka Provides for Our Saga



Durability

Events persisted to disk — if Inventory is down, OrderPlaced waits.



Decoupling

Order Service doesn't know if Inventory is up — just publishes and continues.



Replay

A crashed consumer re-reads from the last committed offset.



Fan-out

Multiple services can consume the same event (Notification + Inventory).

WHAT WE USE IN S7

- order-events — Order Service publishes
- inventory-events — Inventory Service publishes
- payment-events — Payment Service publishes

WHAT WE DO NOT COVER

- Kafka Partitions internals · Exactly-Once Semantics
- Consumer Group rebalancing · Kafka Streams · Schema Registry

Spring Boot Kafka Configuration

Pom.xml — ADD Spring Kafka dependency TO EACH SAGA SERVICE

```
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
```

Spring Boot Kafka Configuration

APPLICATION.YML — ADD TO EACH SAGA SERVICE

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: ...StringSerializer
      value-serializer: ...JsonSerializer
    consumer:
      group-id: ${spring.application.name}
      key-deserializer: ...StringDeserializer
      value-deserializer: ...JsonDeserializer
    properties:
      spring.json.trusted.packages: "com.microservices.pro.*"
      auto-offset-reset: earliest
```

NOTES

- group-id = own consumer group per service — each tracks its own offset
- auto-offset-reset: earliest — new consumers read from the beginning
- trusted.packages required, or JSON deserializer rejects unknown classes
- Kafka already running (Docker Compose since S1) — no setup needed

Define All Domain Events First

```
public record OrderPlacedEvent(  
    String orderId, String productId, int quantity, BigDecimal amount, String customerId) {}  
public record InventoryReservedEvent(String orderId, String productId, int quantity) {}  
public record InventoryReservationFailedEvent(String orderId, String reason) {}  
public record PaymentCompletedEvent(String orderId, String transactionId) {}  
public record PaymentFailedEvent(String orderId, String reason) {}  
public record InventoryReleasedEvent(String orderId) {}  
public record OrderConfirmedEvent(String orderId, String transactionId) {}  
public record OrderCancelledEvent(String orderId, String reason) {}
```



Events are defined on the board AND in code — before any service logic. In production: shared library or events module. In this course: duplicated per service (shared libs discussed Session 14).

Three Services, One Conversation



- We just defined the EVENTS — the vocabulary all three services share
- Next: we wire each service's `@KafkaListener` so it reacts to the right event and publishes the next one
- No service calls another directly. They only talk through Kafka topics.

createOrder() Publishes OrderPlaced

OrderService.java — Publish OrderPlaced event instead of direct service calls

```
@Service
public class OrderService {
    @Autowired private KafkaTemplate<String, Object> kafkaTemplate;
    @Autowired private OrderRepository orderRepository;

    public OrderResponse createOrder(OrderRequest request) {
        // Step 1: Create order in PENDING state (local transaction)
        Order order = new Order(UUID.randomUUID().toString(), request.getProductId(),
            request.getQuantity(), request.getAmount(), OrderStatus.PENDING);
        orderRepository.save(order);

        // Step 2: Publish event to start the Saga (async - no waiting)
        kafkaTemplate.send("order-events", order.getOrderId(),
            new OrderPlacedEvent(order.getOrderId(), request.getProductId(),
                request.getQuantity(), request.getAmount(), request.getCustomerId()));

        // Return immediately - client gets PENDING, not final state
        return new OrderResponse(order.getOrderId(), OrderStatus.PENDING, "Order received...");
    }

    // Listen for Saga completion events
    @KafkaListener(topics = "payment-events", groupId = "order-service")
    public void handlePaymentEvent(String rawJson) {
        // Parse and handle PaymentCompleted or PaymentFailed
        // See full implementation below
    }
}
```

Order Service listens for Saga outcomes

OrderSagaEventHandler.java — handles final Saga outcomes - handlePaymentEvent

```
@Service
public class OrderSagaEventHandler {
    @Autowired
    private OrderRepository orderRepository;

    @KafkaListener(topics = "payment-events", groupId = "order-service")
    public void handlePaymentEvent(
        @Payload String rawEvent,
        @Header(KafkaHeaders.RECEIVED_TOPIC) String topic) {

        // Simplified: parse event type and orderId from JSON
        // In production: use proper JSON deserialization with type discriminator
        if (rawEvent.contains("PaymentCompleted")) {
            PaymentCompletedEvent event = parsePaymentCompleted(rawEvent);
            Order order = orderRepository.findById(event.orderId()).orElseThrow();
            order.setStatus(OrderStatus.CONFIRMED);
            orderRepository.save(order);
            log.info("[SAGA] Order {} CONFIRMED ☐ ", event.orderId());
        } else if (rawEvent.contains("PaymentFailed")) {
            PaymentFailedEvent event = parsePaymentFailed(rawEvent);
            Order order = orderRepository.findById(event.orderId()).orElseThrow();
            order.setStatus(OrderStatus.PAYMENT_FAILED);
            orderRepository.save(order);
            log.warn("[SAGA] Order {} payment failed, waiting for inventory release...", event.orderId());
        }
    }
}
```

Order Service listens for Saga outcomes

OrderSagaEventHandler.java — handles final Saga outcomes - handleInventoryReleased

```
@Service
public class OrderSagaEventHandler {
    @Autowired
    private OrderRepository orderRepository;

    @KafkaListener(topics = "inventory-events", groupId = "order-service-cancel")
    public void handleInventoryReleased(String rawEvent) {
        if (rawEvent.contains("InventoryReleased")) {
            InventoryReleasedEvent event = parseInventoryReleased(rawEvent);
            Order order = orderRepository.findById(event.orderId()).orElseThrow();
            order.setStatus(OrderStatus.CANCELLED);
            orderRepository.save(order);
            log.info("[SAGA] Order {} CANCELLED - inventory released □ ", event.orderId());
        }
    }
}
```

Test It Live: POST /api/orders

```
$ curl -X POST http://localhost:8082/api/orders -d '{"productId":"PROD-001","quantity":3,...}'  
  
{ "orderId": "a1b2c3", "status": "PENDING", "message": "Order received — processing..." }
```

Watch the response: status is PENDING immediately — not CONFIRMED.

- The client does not wait for Inventory or Payment to finish — Order Service returned the moment its own local transaction committed
- Confirmation arrives later, driven entirely by Kafka events — this is the async Saga in action

Reserve Stock — and Compensate on Failure

InventorySagaHandler.java — handles Reserve inventory when order is placed

```
@Service
public class InventorySagaHandler {

    @Autowired
    private InventoryService inventoryService;

    @Autowired
    private KafkaTemplate<String, Object> kafkaTemplate;

    // STEP 2 of Saga: Reserve inventory when order is placed
    @KafkaListener(topics = "order-events", groupId = "inventory-service")
    public void handleOrderPlaced(OrderPlacedEvent event) {
        log.info("[SAGA] Handling OrderPlaced for order: {}", event.orderId());
        try {
            inventoryService.reserveStock(event.productId(), event.quantity(), event.orderId());
            // Success: publish InventoryReserved
            kafkaTemplate.send("inventory-events", event.orderId(),
                new InventoryReservedEvent(event.orderId(), event.productId(), event.quantity()));
            log.info("[SAGA] Inventory reserved for order: {} □ ", event.orderId());
        } catch (InsufficientStockException e) {
            // Failure: publish InventoryReservationFailed (starts compensation)
            kafkaTemplate.send("inventory-events", event.orderId(),
                new InventoryReservationFailedEvent(event.orderId(), e.getMessage()));
            log.warn("[SAGA] Inventory reservation FAILED for order: {} □ ", event.orderId());
        }
    }
}
```

Reserve Stock — and Compensate on Failure

InventorySagaHandler.java — handles COMPENSATION: Release inventory when payment fails

```
@Service
public class InventorySagaHandler {

    @Autowired
    private InventoryService inventoryService;

    @Autowired
    private KafkaTemplate<String, Object> kafkaTemplate;

    // COMPENSATION: Release inventory when payment fails
    @KafkaListener(topics = "payment-events", groupId = "inventory-compensation")
    public void handlePaymentFailed(String rawEvent) {
        if (rawEvent.contains("PaymentFailed")) {
            PaymentFailedEvent event = parsePaymentFailed(rawEvent);
            inventoryService.releaseStock(event.orderId()); // undo reservation
            kafkaTemplate.send("inventory-events", event.orderId(),
                new InventoryReleasedEvent(event.orderId()));
            log.info("[SAGA] COMPENSATION: Inventory released for order: {} □ ", event.orderId());
        }
    }
}
```

Two Consumer Groups, One Service

Consumer Group	Listens To	Responsibility
inventory-service	order-events	Reserve stock when a new order arrives
inventory-compensation	payment-events	Release stock when payment fails (compensation only)



If Inventory used ONE group for both, one logical consumer would have to handle two unrelated jobs on the same offset stream.

- Separate groups isolate the reservation flow from the compensation flow — independent scaling, independent failure handling
- Real bug we guard against: mixing these up makes one consumer process both events and causes duplicate or skipped compensation

Process Payment — or Trigger Compensation

PaymentSagaHandler.java — Process & Fail

```
@Service
public class PaymentSagaHandler {

    @Autowired private PaymentService paymentService;

    @Autowired private KafkaTemplate<String, Object> kafkaTemplate;

    // STEP 3 of Saga: Process payment when inventory reserved
    @KafkaListener(topics = "inventory-events", groupId = "payment-service")
    public void handleInventoryReserved(String rawEvent) {
        if (!rawEvent.contains("InventoryReserved")) return; // ignore other events
        InventoryReservedEvent event = parseInventoryReserved(rawEvent);

        log.info("[SAGA] Processing payment for order: {}", event.orderId());
        try {
            String txId = paymentService.processPayment(event.orderId());
            kafkaTemplate.send("payment-events", event.orderId(),
                new PaymentCompletedEvent(event.orderId(), txId));
            log.info("[SAGA] Payment COMPLETED for order: {} ☐ ", event.orderId());
        } catch (PaymentException e) {
            kafkaTemplate.send("payment-events", event.orderId(),
                new PaymentFailedEvent(event.orderId(), e.getMessage()));
            log.warn("[SAGA] Payment FAILED for order: {} ☐ - triggering compensation", event.orderId());
        }
    }
}
```

Watch the Log Sequence Across All 3 Services

Happy Path — stock available, payment succeeds

```
[SAGA] Inventory reserved ☐  
[SAGA] Payment COMPLETED ☐  
[SAGA] Order CONFIRMED ☐
```

Compensation — set failure rate to 100%

```
[SAGA] Inventory reserved ☐  
[SAGA] Payment FAILED ☐ — triggering compensation  
[SAGA] COMPENSATION: Inventory released ☐  
[SAGA] Order CANCELLED — inventory released ☐
```

Distributed Business Transactions — Online

📋 Capability Added

Distributed Business Transactions

A payment failure no longer leaves the system inconsistent — inventory is released and the order is cancelled automatically.

📋 Services Added / Updated

- NEW: Kafka event bus — order/inventory/payment-events
- UPDATED: Order, Inventory, Payment Services — Saga handlers

Enterprise E-Commerce Platform — After Session 7



Config+Eureka



Gateway Secured



Product Svc



Resilience Full



Feign Sync



Order Svc Saga



Kafka Running



Notif. Svc



DAILY QUIZ

8 Questions · 10 Minutes

Business Inconsistency · 2PC · Saga · Choreography · Compensation · Kafka Durability · Consumer Groups

Homework & Checkpoint Commit

DEFINITION OF DONE — SESSION 7

- POST /api/orders returns PENDING + orderId
- GET /api/orders/{id}/status returns CONFIRMED (happy path)
- Compensation: failure rate 100% → order CANCELLED
- All 3 Kafka topics receiving events (check logs)
- [SAGA] COMPENSATION log visible on payment failure
- 5 unit tests passing: mvn test green
- Commit: session-07: add-choreography-saga

KNOWLEDGE CHECK — BEFORE SESSION 8

Topic: Redis Caching Strategies

- What is the difference between Cache-Aside and Write-Through caching?
- When would you invalidate a cache entry in a microservices system?
- What is a TTL (Time-to-Live) and what problem does it solve?

Also: build the bonus Notification Service — consumes PaymentCompleted, logs “Email sent”

5 Real Errors, 5 Real Fixes

Issue	Solution
Consumer never receives events	Check <code>spring.kafka.consumer.group-id</code> is unique per service. Verify <code>auto-offset-reset</code> : earliest.
ClassCastException deserializing events	Add <code>spring.json.trusted.packages: "com.microservices.pro.*"</code> to consumer config.
Compensation fires but stock not released	<code>inventory-compensation</code> must be a DIFFERENT consumer group from <code>inventory-service</code> .
Order stays PENDING indefinitely	Kafka consumer not running — check startup logs. Verify topic names match exactly.
Duplicate compensation (released twice)	Add idempotency check: if order is already CANCELLED, skip compensation.



SESSION 7 COMPLETE

First Distributed Saga Running

Order → Inventory → Payment, with full compensation on failure

NEXT SESSION

Session 8 — Caching Strategies + Architecture Clinic #1

WHEN

Monday, 3:00–5:30 PM, Online